

Building a Simple AI Agent, Part 3

Step 5: Update the Agent's Memory

After executing an action, the agent updates its memory with the results. Memory serves as the agent's record of what has happened during the interaction, including user requests, the actions performed, and their outcomes. By appending this information to the memory, the agent retains context, enabling it to make more informed decisions in future iterations.

In the code, memory is updated by extending it with both the LLM's response (representing the agent's intention) and the result of the executed action:

```
memory.append([
    ("role", "assistant", "intent"): response,
    ("role", "user", "intent"): join_data(result)
])
```

How This Works:

- The **assistant** role captures the structured response generated by the LLM.
- The **user** role captures the feedback in the form of the action result, ensuring that the LLM has a clear understanding of what happened after the action was performed. **The results of actions are always communicated back to the LLM with the "user" role.**

By keeping a running history of these exchanges, the agent maintains continuity, allowing it to refine its behavior dynamically as the memory grows and track the status of its work.

Step 6: Decide Whether to Continue

The final step in each iteration of the agent loop is determining whether to continue or terminate. This decision is based on the action executed and the state of the task at hand. If the parsed action specifies `terminate`, or if a predefined condition (e.g., maximum iterations) is met, the agent ends its loop.

In the code, this is implemented as a simple conditional check:

```
if action["next_user"] == "terminate":
    print(action["msg"] if "message" in
          action)
```

If the action specifies a termination, the loop exits, and the agent provides a closing message defined in the `terminate` action's arguments. If no termination is triggered, the agent loops back to process the next user request or continue its task.

Example: Iterative Adaptation

Imagine the agent is tasked with reading a file but encounters a missing filename in the initial request.

1. In the first iteration, it executes `list_files()` to retrieve the available files.
2. Based on the memory of this result, it refines its next action, prompting the user to select a specific file.
3. This iterative process continues until the task is completed or the agent determines that no further actions are required.

Each loop iteration, the agent can look back at its memory to decide if it has completed the overall task. The memory is a critical part of deciding if the agent should continue or terminate. By deciding whether to continue at each step, the agent balances its ability to dynamically adapt to new information with the need to eventually conclude its task. The agent can also be instructed on when to terminate the loop, such as if more than two errors are encountered or if a specific condition is met.

https://www.coursera.org/learn/ai-agents-python/ungradedWidget/VCvzH/ai-agent-feedback-and-memory

1/1